

程序设计

讲义

2026 年秋季

毕朝日

复旦大学集成电路与微纳电子创新学院

`zhaori_bi@fudan.edu.cn` `https://bzs2915.github.io/bzs2915/`

目录

第二讲 基本数据类型与格式化输入输出	4
2.1 从一个换算程序开始	6
2.1.1 变量、常量与数据的读写	7
2.1.2 输入为什么多一个取地址符	7
2.2 为什么要有数据类型	8
2.2.1 65 还是字母 A	8
2.2.2 整数、小数与字符	8
2.2.3 声明、定义、初始化与赋值	9
2.3 数据在内存中的存储与地址	10
2.3.1 位、字节与地址	10
2.3.2 容量单位	11
2.3.3 怎样知道一个类型占多少空间	12
2.4 进位计数制与 C 中的整数写法	12
2.4.1 位权：进制转换的依据	12
2.4.2 由其他进制转成十进制	13
2.4.3 由十进制整数转成其他进制	13
2.4.4 二进制与八进制、十六进制的直接转换	14
2.4.5 常量前缀与输出格式	14
2.5 有符号整数与补码	15
2.5.1 原码：用一位表示符号	15
2.5.2 反码及其运算中的问题	16
2.5.3 补码：用加法完成减法	16
2.5.4 补码的取值范围为什么不对称	17
2.5.5 增加位数时的符号扩展	17
2.6 C 语言中的整数类型	18
2.6.1 整数类型的长度与符号	18
2.6.2 常量本身也有类型	19
2.6.3 整数常量的后缀	19
2.6.4 有符号溢出与无符号回绕	20

2.6.5	整数输出与长度修饰符	21
2.7	浮点数：范围与精度	22
2.7.1	小数点放在哪里	22
2.7.2	三种浮点类型	23
2.7.3	浮点常量的写法与后缀	24
2.7.4	类型转换与赋值结果	24
2.7.5	加上的 1 到哪里去了	25
2.7.6	浮点数输出	26
2.8	字符、编码与转义序列	26
2.8.1	用编号表示字符	26
2.8.2	单引号、双引号与变量名	27
2.8.3	字符可以参与整数运算	28
2.8.4	用转义序列表示控制字符	28
2.8.5	八进制与十六进制转义	30
2.8.6	一个字符会占几个字节	30
2.9	格式化输出：把值变成文本	31
2.9.1	格式字符串与参数的对应关系	31
2.9.2	字段宽度、精度与百分号	31
2.9.3	三类参数不匹配	32
2.10	格式化输入与交互过程	33
2.10.1	将输入文本转换为数值	33
2.10.2	检查输入是否成功	33
2.10.3	空白、普通字符与字符输入	34
2.10.4	月薪计算：观察控制字符的作用	35
2.10.5	输出何时真正出现	35
2.11	习题与解析	36
2.11.1	习题一：判断常量写法	36
2.11.2	习题二：常量的类型、意义与转义	37
2.11.3	习题三：修正税额计算程序	37
2.11.4	习题四：由牛腿数计算牛的数量	38
2.11.5	习题五：变量类型与输出格式	39
2.11.6	习题六：前缀、后缀与类型选择	40
2.11.7	习题七：同一位串的解释与扩展	40
2.11.8	习题八：计算精度与输出精度	41
2.11.9	习题九：连续输入与剩余字符	41
2.12	本讲总结	42

第二讲 基本数据类型与格式化输入输出

1970 年，贝尔实验室的 UNIX 小组有了一台 PDP-11 计算机。汤普森和里奇准备把已有的程序移到这台新机器上，他们使用的 B 语言却遇到了一个麻烦：机器能够直接读写单个字符，语言中的普通取数和赋值操作却仍以整个机器字为单位。

一个二进制数字只能取 0 或 1，所占的存储单位称为**位**（bit，binary digit 的缩写）。在 PDP-11 上，一个**字节**（byte）是 8 位，一个**机器字**（word，机器一次处理的基本数据单位）是 16 位，也就是两个字节。

字母存入内存时，用约定的数字编号表示，这个编号称为字符的**编码值**。PDP-11 上的一个英文字母编码用一个字节保存，一个机器字便可以存下两个字母。

存储单元的编号称为**地址**，按编号找到数据称为**寻址**。B 最初运行的机器为每个机器字分配地址，语言也只提供一种基本数据单位：机器字。整数用一个字保存，地址也用一个字保存。

一串连续的字符称为**字符串**。假设内存中存着字符串 ABCD，现在要把第二个字母 B 改成 X，得到 AXCD。B 把字符紧挨着存放，让几个字符共用一个机器字，以节省空间。图中只画出四个字母，格子按内存地址从小到大排列；每个小格是一个字节，相邻两个小格组成一个机器字。

byte address	1000	1001	1002	1003
	+-----+-----+-----+-----+			
contents	A	B	C	D
	+-----+-----+-----+-----+			
	<---- word 0 ----->		<---- word 1 ----->	
B: s[0]	= the whole word containing A and B			
B: s[1]	= the whole word containing C and D			
char index	0	1	2	3
		^		
	change this byte			

图中的 s 表示这段字符串的起始位置。方括号表示按编号取元素，编号从 0 开始，所以 s[0] 取第一个元素，s[1] 取第二个元素。问题在于，B 的一个元素是一个机器字。在图中，s[0] 一次取出 A 和 B 所在的整个字，s[1] 取出的已经是 C 和 D 所在的

字。因此，要修改字母 B，不能直接把 `s[1]` 当作第二个字符来赋值；那样会改写后面整个机器字的内容。

B 用专门的库函数读取和修改单个字符。`char(s, 1)` 表示取出字符串中编号为 1 的字符，`lchar(s, 1, 'X')` 表示把这个字符改成 X。这两个函数见于汤普森的 PDP-11 版 B 语言手册。两个 B 库函数根据字符串的起始位置和字符编号定位字节。对于从地址 1000 开始的 ABCD，编号 1 的字符在地址 1001，也就是第一个机器字的第二个字节。函数调用 `char(s, 1)` 中的 `char` 是 B 的函数名。

PDP-11 已经能够给每个字节单独编号，也有读写字节的机器指令。只要知道目标地址是 1001，就可以修改那一个字节，并保留其他三个字母。B 的字符库函数内部也可以利用这些指令；不便之处在于，程序员在语言中仍要区分两套操作：取整个字用下标，取其中的字符要调用专门函数。

如果编译器知道 `s` 中每个元素都是一个字节，普通的下标操作就可以直接表示某个字符。后来 C 语言中的字符类型提供了这种区分。用今天的 C 写法 `char s[] = "ABCD"`；就是定义一个字符数组，即一组连续存放的字符元素。数组 `s` 仍从地址 1000 开始存放，所以 `s[1]` 对应地址 1001，表示字母 B。下面只列出读写字符的相关语句，`c` 是接收字符值的变量。左边假定 B 字符串 `s` 已经存在，右边先用 C 定义一个可以修改的字符数组。

B: word-based elements	C: char elements
-----	-----
	<code>char s[] = "ABCD";</code>
<code>c = char(s, 1);</code>	<code>c = s[1];</code>
<code>lchar(s, 1, 'X');</code>	<code>s[1] = 'X';</code>
v	v
library locates the byte	compiler uses char size
+-----+	+-----+
v	
write X to address 1001	
before:	
+-----+	
A B C D	
+-----+	
v	
after:	
+-----+	
A X C D	
+-----+	

C 的类型声明让编译器掌握了元素的大小：字符数组每前进一个元素，地址就增加 1 字节；在 PDP-11 上，整数数组每前进一个元素，地址则增加 2 字节。同样写 `s[1]`，编译器可以根据数组的类型算出相应地址，并选择合适的读写指令。程序员可以用一致的下标和赋值写法处理不同类型的数据。

浮点数用有效数字部分和指数共同表示数值，例如把 6.22 写成 6.22×10^0 ，把 622 写成 6.22×10^2 。假设一种浮点格式需要 32 位，它就要占用两个 16 位机器字。原先把每个普通数据都放在一个字中的规则，无法直接表示这样的数，也无法仅凭同一种运算规则处理整数和浮点数。虽然最初的 PDP-11 还没有浮点运算硬件，厂家已承诺提供，语言需要为它作准备。

1971 年，里奇开始给 B 增加字符类型，同时改写编译器，让它直接生成 PDP-11 机器指令。这个过渡版本名为 NB，即 new B，已经区分 `int` 和 `char`，后来逐步发展成 C 语言。他在 1993 年的回顾文章中说明，字符处理、字节寻址和未来的浮点硬件，都是推动这次改造的具体原因。

今天写下 `int n;` 或 `char ch;`，就是在告诉编译器变量的类型，以确定存储空间和运算规则。

2.1 从一个换算程序开始

假设要编写一个美元兑人民币的换算程序，按 1 美元兑换 6.22 元人民币计算。第一讲的示例把参与计算的数直接写在源代码里，换一个数就得修改程序。这次让使用者从键盘输入美元金额，由程序算出相应的人民币金额：

```
#include <stdio.h>

int main(void) {
    float rmb;
    int dollar;

    printf("Enter the money to change (dollar): ");
    fflush(stdout);
    if (scanf("%d", &dollar) != 1) {
        return 1;
    }
    rmb = dollar * 6.22;
    printf("The total money is RMB %.2f\n", rmb);
    return 0;
}
```

输入 100 后，屏幕上会显示 `The total money is RMB 622.00`。换算本身只需做一次乘法，但要让别人用上这个程序，还得先显示提示语，读入金额，算出结果，最后按合适的格式输出。金额通常保留两位小数，所以输出结果时用了 `%.2f`。如果改用 `%f`，小数点后默认显示六位，屏幕上的结果就会变成 `622.000000`。

2.1.1 变量、常量与数据的读写

程序需要分别保存输入的金额和换算结果，因此定义了 `dollar` 与 `rmb` 两个变量。变量有名称，也有用于存放数据的内存空间；程序运行时，其中保存的值可以改变。使用者输入 100 后，`dollar` 保存 100；若在输入语句后执行 `dollar = 200;`，同一个变量保存的值就改为 200。表达式中的 6.22 则是直接写出的固定数值，称为常量。换算程序用 `int` 类型的 `dollar` 保存整美元金额，用 `float` 类型的 `rmb` 保存带小数的换算结果。例如，输入 1 美元，换算结果就是 6.22 元。

输入的 100 是怎样参与计算的？按下数字键时，输入的是 1、0、0 三个字符。`scanf` 按照十进制整数的读入规则，将它们转换为数值 100，存入 `dollar`。把三个输入字符转换为数值 100，可以理解为计算 $1 \times 100 + 0 \times 10 + 0 = 100$ 。随后，程序计算 `dollar * 6.22`，把结果存入 `rmb`；`printf` 再把 `rmb` 中保存的结果转换成 `622.00` 这样的字符，输出到屏幕。

输入时要把字符转换为数值，输出时又要把数值转换为字符。`scanf` 和 `printf` 分别完成这两项工作。函数调用中的 `%d` 指定十进制整数转换，`%f` 指定浮点数转换；这种以百分号开头、指定转换方式的写法称为格式说明。代码中的 `\n` 表示换行。

2.1.2 输入为什么多一个取地址符

同样是使用变量 `dollar`，为什么输入时要在变量名前加 `&`，输出时却不用？比较下面两行代码：

```
scanf("%d", &dollar);
printf("%d\n", dollar);
```

`printf` 只需要知道 `dollar` 当前的值，就能把它打印出来。`scanf` 的任务是修改这个变量，因此还必须知道它在内存中的位置。`&` 称为取地址运算符，`&dollar` 表示变量 `dollar` 的地址。可以把值与地址的区别想成余额与账号的区别：告诉别人账户里有多少钱，报出余额即可；要往账户里存钱，就需要提供账号。输入函数有了地址，才能把读入的数据存到指定的变量中。

使用者也可能输错，比如输入了 `abc`。这些字符无法转换为十进制整数，`dollar` 便没有得到可供本次计算使用的值。为此，程序用 `if` 检查 `scanf` 的返回值。返回值是函数调用结束时交给调用者的结果。成功读入一个数据并赋给变量时，`scanf` 返回 1；!=

表示“不等于”，检查发现返回值不等于 1 时，程序就执行 `return 1;`，结束运行。只有 `scanf` 成功给 `dollar` 赋值，程序才进行换算。

返回 1 并不保证整行输入都是合法整数。例如，输入 100abc 或 100.5 时，`%d` 都可以读取开头的 100 并返回 1，余下的 abc 或 .5 仍留在输入中。开头的换算程序只检查是否读到了一个整数，没有检查后续字符。

`stdout` 表示**标准输出**，在通常的交互运行中连接屏幕。输出文字可能先存放在一块临时内存中，这块内存称为**缓冲区**。`fflush(stdout)` 把其中尚未送出的内容送往输出设备，这个操作称为**刷新**。换算程序的提示语末尾没有换行，主动刷新可以使提示文字在程序等待输入前显示出来。

2.2 为什么要有数据类型

2.2.1 65 还是字母 A

内存中有一串二进制数字 0100 0001。如果把它看作**无符号整数**，即只表示零和正数的整数，它表示 65；如果按照 ASCII（American Standard Code for Information Interchange，美国信息交换标准代码）编码表查找，它对应字母 A。同一串 0 和 1 可以表示数值，也可以表示字符，解释时必须知道所用的表示方法。

在 C 程序中，**数据类型规定了数据的表示方法和运算规则**。整数 100 和浮点数 100.0 虽然数值相等，在内存中的表示却不同。计算 `dollar * 6.22` 时，编译器需要根据两个操作数的类型进行转换，再完成浮点乘法。类型还关系到存储空间的大小：记录学生人数可以用整数，保存电压测量值则往往需要浮点数。例如，`int count` 声明保存整数人数的变量，`double voltage` 声明保存浮点电压值的变量。

2.2.2 整数、小数与字符

有符号整数可以表示负数、零和正数；**无符号整数**只表示非负数。浮点类型用于表示带小数的数和较大范围的数值，字符类型用于保存字符编码。表中按用途整理常用类型，类型名采用常见的简写形式。布尔类型用于保存真假，0 表示假，1 表示真。

表 2.1: 常用基本数据类型（按 C11 整理）

类别 / 用途	类型名称	典型数据
有符号整数	<code>signed char</code> 、 <code>short</code> 、 <code>int</code> 、 <code>long</code> 、 <code>long long</code>	温度、余额变动
无符号整数	<code>unsigned char</code> 、 <code>unsigned short</code> 、 <code>unsigned int</code> 、 <code>unsigned long</code> 、 <code>unsigned long long</code>	计数、字节值
浮点数	<code>float</code> 、 <code>double</code> 、 <code>long double</code>	电压、时间、比值
字符	<code>char</code>	字母、标点、控制字符
布尔值	<code>_Bool</code> ；包含 <code><stdbool.h></code> 后可写 <code>bool</code>	条件是否成立

`char`、`signed char`、`unsigned char` 是三种不同的类型；普通 `char` 采用有符号还是无符号的表示方式，由实现决定。`char` 和 `_Bool` 在 C 的分类中也属于整数类型。`signed` 表示有符号，`unsigned` 表示无符号；`short`、`int`、`long`、`long long` 默认都有符号。本讲示例按 C11 编写，C11 指 2011 年发布的 C 语言标准版本。

`void` 表示没有值，不能用来定义普通变量。在已经见过的 `int main(void)` 中，括号内的 `void` 说明函数没有参数，前面的 `int` 说明函数返回整数；如果把 `void` 写在函数的返回类型位置，则说明函数不返回值。

基本类型以外的数据组织方式（选读）

一个变量保存单个数值时，可以直接使用表中的类型。保存多项数据则需要组织方式：数组连续存放一组同类型的数据，例如开头的字符数组 `s`；结构体把不同类型的成员放在一起，例如一名学生的编号和成绩；联合体让成员共用一块存储空间；枚举为整数值命名。指针用于保存地址，`scanf` 接收的 `&dollar` 就是指向变量 `dollar` 的指针值。

2.2.3 声明、定义、初始化与赋值

声明向编译器说明名称和类型；变量的定义还为它提供存储空间。例如，`int row`；既是声明，也是定义。定义时可以同时给出初值，也可以在后面的计算或输入语句中赋值：

```
int row, col, goat;
int ss = 20;
int dog, cat = 30;
row = 1;
```

代码片段假定写在函数体内，row、col、goat、ss、dog 和 cat 都是**局部变量**。定义 ss 时用 = 20 给出初值，称为**初始化**。row = 1; 把 1 存入已经定义好的 row，称为**赋值**。

定义语句 int dog, cat = 30; 只初始化了 cat。row、col、goat 和 dog 在各自的定义语句中都没有初始化；执行 row = 1; 后，row 有了值，col、goat 和 dog 仍须赋值后才能读取。

如果三个计数器都需要从 0 开始，应把定义写成 int row = 0, col = 0, goat = 0;。每个变量都有自己的初始化部分，三个变量在定义时就获得了初值 0。变量在定义语句中的先后顺序不代表内存中的排列顺序；具体位置由编译器安排。

C 要求先声明变量，再使用变量，并且声明在使用处必须可见。初学时常把变量集中定义在函数体开头，便于查看程序用到了哪些数据。C99（1999 年发布的 C 语言标准）及后续版本也允许在语句之间定义变量，因此还可以把定义和初始化写在第一次使用变量的附近。

2.3 数据在内存中的存储与地址

2.3.1 位、字节与地址

每一位只能取 0 或 1。在常见的计算机中，8 位组成一个字节：

$$1 \text{ byte} = 8 \text{ bits}, \quad 2^8 = 256.$$

每一位都可以取 0 或 1，8 位就有 256 种组合。如果用这些组合表示非负整数，能够表示的范围就是 0 至 255。若还要表示负数，就需要把其中一部分编码分配给负数。

按字节编址的存储器为每个字节分配一个编号，这个编号就是**地址**。假设一个 int 占 4 字节，变量 dollar 的起始地址为 1000，那么它占用的四个字节的地址依次是 1000、1001、1002、1003，&dollar 表示起始地址 1000。调用 scanf 时，地址指定数据写入的位置，%d 则说明应当按照 int 类型进行转换和存储。

字地址与字节地址（选读）

在 PDP-11 上，一个机器字占两个字节。早期 B 程序中的地址按机器字计数，硬件地址按字节计数：B 程序中的字地址 500，对应硬件的字节地址 1000。运行时需把字地址乘以 2，才能交给硬件使用。里奇后来把语言中的地址改为按字节表示，再由类型决定访问的数据有多大。

地址也用二进制表示。32 位可以组成 2^{32} 个不同的地址，每个地址对应一个字节时，总共能够为 2^{32} 个字节编号，这个可编号的范围称为**地址空间**。64 位能够表示的地址更多，但地址位数不能直接说明一台计算机装了多少内存。实际安装的内存称为**物理内存**；程序使用的地址可以由处理器和操作系统映射到物理内存，这类地址称为**虚拟地址**，其可用范围还受到处理器和操作系统的限制。

历史旁注

1964 年 4 月 7 日，IBM 发布 System/360 系列计算机。在此之前，IBM 的多条产品线各有自己的体系，用户更换机器常常也要改写软件。System/360 把性能不同的型号纳入同一套兼容体系，使用户能够在不同型号上继续使用同一套软件。System/360 采用 8 位字节，推动了 8 位字节的普及。一个 8 位字节有 256 种取值，足以容纳当时常用的单字节字符编码。

2.3.2 容量单位

容量单位中的 B 表示 byte（字节）。十进制前缀 kilo、mega、giga、tera 分别表示 10^3 、 10^6 、 10^9 、 10^{12} 倍，配上 B 就得到 kB、MB、GB、TB。

二进制容量采用 kibi、mebi、gibi、tebi 四个前缀，分别表示 2^{10} 、 2^{20} 、 2^{30} 、 2^{40} 倍。名称取相应十进制前缀的前两个字母，再加上来自 binary（二进制）的 bi，例如 kilo 变成 kibi。符号中的 i 用来区分二进制前缀，不是一个独立英文单词的缩写。四个单位的全称和换算关系如下：

表 2.2: 常用二进制容量单位与十进制容量单位

二进制单位	英文全称	字节数	十进制单位
1 KiB	kibibyte	$2^{10} = 1024$	1 kB = 10^3 字节
1 MiB	mebibyte	$2^{20} = 1048576$	1 MB = 10^6 字节
1 GiB	gibibyte	$2^{30} = 1073741824$	1 GB = 10^9 字节
1 TiB	tebibyte	2^{40}	1 TB = 10^{12} 字节

一块标称容量为 1 GB 的存储设备，换用二进制单位表示，容量为

$$\frac{10^9}{2^{30}} \text{ GiB} \approx 0.9313 \text{ GiB}.$$

这块存储设备格式化后，文件系统还会占用少量空间，可供用户存放文件的容量会进一步减少。有些旧书和软件界面仍用 KB、MB、GB 表示二进制容量，比较数字时应先确认单位定义。

容量单位也可用于描述地址空间。按字节编址时，32 位地址能够为 2^{32} 个字节编号，即 4 GiB。4 GiB 说明可以区分多少个字节位置，不表示机器实际安装了同样多的内存。

2.3.3 怎样知道一个类型占多少空间

同一个类型在不同的编译环境中可能占用不同大小的空间。`sizeof` 运算符给出类型或对象（例如变量）占用的字节数。结果的类型是 `size_t`，这是 C 专门用于表示大小的无符号整数类型。

输出 `size_t` 值要用 `%zu`：% 开始一项转换说明，`z` 是长度修饰符，在 `%zu` 中指定参数类型为 `size_t`；`u` 表示按无符号十进制输出。`z` 是 C 为这类大小值规定的修饰符，可联系 `size` 来记忆；它不表示固定的字节数，而随当前环境中 `size_t` 的实际类型确定。`<limits.h>` 是提供整数类型边界信息的头文件，其中的 `CHAR_BIT` 给出一个 C 字节包含的位数。

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("bits per byte: %d\n", CHAR_BIT);
    printf("char: %zu, short: %zu, int: %zu\n",
           sizeof(char), sizeof(short), sizeof(int));
    printf("long: %zu, float: %zu, double: %zu\n",
           sizeof(long), sizeof(float), sizeof(double));
    return 0;
}
```

在常见的 Windows 编译环境中，输出可能是 `char: 1, short: 2, int: 4` 和 `long: 4, float: 4, double: 8`。换一种编译环境，`long` 也可能占 8 字节。

C 规定 `char` 的大小为 1 字节，所以 `sizeof(char)` 的结果始终是 1。C 要求一个字节至少有 8 位；大多数常用计算机恰为 8 位，也有机器采用更大的字节。`CHAR_BIT` 属于宏：预处理器在编译前把这个名称替换为头文件定义的内容。

2.4 进位计数制与 C 中的整数写法

2.4.1 位权：进制转换的依据

十进制中，同一个数字写在个位、十位和百位上，表示的数值各不相同。这种由位置决定的权重称为位权。例如，百位的位权是 10^2 ，二进制从右起第三位的位权则是 2^2 。一般来说， B 进制使用 0 至 $B-1$ 这 B 个数码，逢 B 进一。整数 X 若写成 $(K_{n-1}K_{n-2}\cdots K_0)_B$ ，其值就是各位数码与位权乘积的和：

$$X = \sum_{i=0}^{n-1} K_i B^i = K_{n-1} B^{n-1} + \cdots + K_1 B + K_0.$$

小数点右侧的各位也有位权，只是指数变为负数。例如：

$$(123.1)_{10} = 1 \times 10^2 + 2 \times 10 + 3 + 1 \times 10^{-1}.$$

十进制使用 0 至 9，二进制只使用 0 和 1，八进制使用 0 至 7。十六进制需要十六个数码，除了 0 至 9，还用 A 至 F 依次表示 10 至 15；这些字母也可以写成小写 a 至 f。

2.4.2 由其他进制转成十进制

把各位数码乘以相应的位权，再将结果相加，就能把其他进制的数转换成十进制数：

$$(10111)_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2 + 1 = 23,$$

$$(261)_8 = 2 \times 8^2 + 6 \times 8 + 1 = 177,$$

$$(F261)_{16} = 15 \times 16^3 + 2 \times 16^2 + 6 \times 16 + 1 = 62049.$$

计算 $(10111)_2$ 时，可以只累加非零位的位权，写成 $16 + 4 + 2 + 1 = 23$ 。手算时不妨先估计一下数值范围：五位二进制整数最大为 31，如果算出了一个三位十进制数，就说明某一步出了错。

2.4.3 由十进制整数转成其他进制

把十进制整数 37 转换成二进制，可以从最低位开始求。37 除以 2，商 18、余 1，说明 $37 = 18 \times 2 + 1$ ，最低位应当是 1。再用商 18 除以 2，余数为 0，这就是倒数第二位。继续用所得的商除以 2，直到商为 0，便能逐次求出各位数码。由于最低位最先求出，书写结果时要把余数倒过来排列。这种方法称为除基取余法。

表 2.3: 将十进制 37 转换为二进制

被除数	除以 2 的商	余数
37	18	1
18	9	0
9	4	1
4	2	0
2	1	0
1	0	1

从表格底部向上读余数，得到 $(100101)_2$ 。再按位权算回去， $32 + 4 + 1 = 37$ ，结果相符。同样的方法也适用于八进制和十六进制，只需改变除数： $37 = 4 \times 8 + 5$ ，所以 37 的八进制写法是 $(45)_8$ ； $37 = 2 \times 16 + 5$ ，所以十六进制写法是 $(25)_{16}$ 。

2.4.4 二进制与八进制、十六进制的直接转换

三位二进制有 $2^3 = 8$ 种组合，恰好对应一个八进制数码；四位二进制有 $2^4 = 16$ 种组合，恰好对应一个十六进制数码。转换整数时，从右向左把二进制数字分组，再逐组替换即可。最左边的一组位数不足时，在前面补 0：

$$(1011)_2 = (001\ 011)_2 = (13)_8,$$

$$(10011)_2 = (0001\ 0011)_2 = (13)_{16}.$$

两个结果虽然都写作 13，数值却不同：八进制的 13 等于十进制 11，十六进制的 13 等于十进制 19。书写时标明进制，才能避免混淆。

表 2.4: 四位二进制与十六进制的对应

二进制	十六进制	二进制	十六进制
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

芯片的寄存器用于保存少量数据或控制信息，手册常用十六进制列出它们的值。0x80、0xFF 开头的 0x 就是 C 中的十六进制前缀。十六进制的 80 对应二进制 1000 0000，FF 对应 1111 1111。用两个数码就能写出一个字节的内容，需要查看某一位时又可以立即展开成二进制，阅读和抄写都比较方便。这也是底层编程经常使用十六进制的原因。

2.4.5 常量前缀与输出格式

C 源程序用前缀区分整数常量的进制：十进制整数通常直接书写，八进制整数以 0 开头，十六进制整数以 0x 或 0X 开头。因此，63 表示十进制 63，063 表示十进制 51，0x63 表示十进制 99。09 是不合法的八进制整数常量，因为八进制中没有数码 9。书写整数常量时，不能为了对齐就在前面补 0，否则编译器会按八进制解释常量。

整数常量 365u 末尾的 u 来自 unsigned，表示采用无符号类型；365 能用 unsigned int 表示，因此 365u 就是这个类型。下面分别按三种进制输出 365，%u、%o、%x 依次指定十进制、八进制和十六进制：

```
#include <stdio.h>

int main(void) {
    unsigned int x = 365u;
    printf("dec = %u; octal = %o; hex = %x\n", x, x, x);
    printf("dec = %u; octal = %#o; hex = %#x\n", x, x, x);
    return 0;
}
```

输出为:

```
dec = 365; octal = 555; hex = 16d
dec = 365; octal = 0555; hex = 0x16d
```

第二个 `printf` 调用使用了 `#` 标志，八进制结果带有前缀 `0`，十六进制结果带有前缀 `0x`。若希望十六进制字母采用大写，可以改用 `%X`。输出十进制整数时，有符号类型用 `%d`，无符号类型用 `%u`，都不使用 `#` 标志。

无论采用哪种输出格式，变量 `x` 保存的数值都是 365。365、0555 和 0x16d 只是这个数值的不同写法。在源程序中，前缀帮助编译器识别常量的进制；在输出语句中，格式说明决定屏幕上采用哪种写法。

2.5 有符号整数与补码

2.5.1 原码：用一位表示符号

如果换算程序还要记录账户余额的变动，就需要用到负数：增加 11 元记作 11，减少 11 元记作 -11 。纸上可以直接写负号，计算机却必须用 0 和 1 表示正负。一个直观的办法是把最高位作为符号位，0 表示正，1 表示负，其余各位表示绝对值。这种表示方法称为**原码**。以 8 位为例：

+1 : 0000 0001, -1 : 1000 0001.

把这两个编码直接送进二进制加法器，0000 0001 加 1000 0001 得到 1000 0010，按原码解释为 -2 ，不等于算术结果 0。要用原码正确完成有符号数的加法，电路还得判断符号、比较绝对值，再决定执行加法还是减法。

原码还有两种零：0000 0000 表示正零，1000 0000 表示负零。同一个零占用了两个编码，判断一个数是否为零时，也必须考虑两种情况。

2.5.2 反码及其运算中的问题

另一种方案是保留正数的编码，把对应正数的各位取反（0 变 1、1 变 0）来表示负数，这称为反码。在 8 位反码中，+1 仍为 0000 0001，-1 则为 1111 1110。两者相加得到 1111 1111，按反码规则解释为负零。

采用反码，正负数可以使用同一套加法电路，但运算仍需要特殊处理：最高位如果产生进位，还要把这个进位加回最低位。正零与负零也仍然各占一个编码。

2.5.3 补码：用加法完成减法

时针现在指向 8 点，要回到 3 小时前的 5 点，可以逆时针拨 3 格，也可以顺时针拨 9 格。两种拨法相差一整圈，最后指向同一个位置。用数学符号表示，就是

$$8 - 3 \equiv 8 + 9 \pmod{12}.$$

式中的 $\equiv$ 表示两边对 12 取模后余数相同， $\pmod{12}$ 说明模数是 12。取模就是按给定的模数求余数，例如 17 对 12 取模得到 5。

8 位二进制共有 256 种组合，也可以看作一个循环。两个 8 位数相加，如果只保留结果的低 8 位，就相当于对 256 取模。因此，可以用 $256 - 1 = 255$ 的编码 1111 1111 表示 -1。再计算 $1 + (-1)$ ：

$$\begin{array}{r} 0000\ 0001 \\ +\ 1111\ 1111 \\ \hline 1\ 0000\ 0000 \end{array}$$

舍去第九位的进位，留下的 8 位全是 0，与 $1 + (-1) = 0$ 一致。推广到 n 位，负整数 $-x$ ($1 \leq x \leq 2^{n-1}$) 用 $2^n - x$ 的二进制编码表示，非负数沿用普通二进制写法，这就是补码的表示规则。采用补码后，普通的二进制加法器就能用于有符号数的加减运算。

求负数的补码时，通常先把它的绝对值写成 n 位二进制，再逐位取反，加 1。例如，求 -11 的 8 位补码：

$$11 : 0000\ 1011 \longrightarrow 1111\ 0100 \longrightarrow 1111\ 0101.$$

8 位全为 1 按无符号数解释为 255，将 11 的各位取反得到 $255 - 11$ ，再加 1 就得到 $256 - 11 = 245$ 。例如余额增加 20 元后减少 11 元，可以把 20 与 -11 的补码按普通二进制加法相加；按无符号数计算就是：

$$20 + 245 = 265 = 256 + 9.$$

只保留低 8 位，相当于减去 256，结果为 9，与 $20 - 11$ 一致。8 位补码 1111 0101 代表 -11，245 只是把同一组位按无符号整数读出的值。

2.5.4 补码的取值范围为什么不对称

把 n 位位串按无符号整数读出的值记为 u 。最高位为 0 时，补码沿用非负数的二进制写法，数值就是 u 。最高位为 1 时，位串表示负数：编码规则把负数 $-a$ 写成 $2^n - a$ ，所以反过来读出数值时应计算 $u - 2^n$ 。

最高位为 1 的无符号值可以拆成“最高位的 2^{n-1} ”与“其余各位之和”。减去 2^n 后，

$$u - 2^n = (2^{n-1} + \text{其余各位之和}) - 2^n = -2^{n-1} + \text{其余各位之和}.$$

因此，按补码读数时，最高位的位权为 -2^{n-1} ，其余位权不变。若从右到左的各位数码为 $b_0, b_1, \dots, b_{n-1}$ ，每个数码为 0 或 1，补码表示的数值 x 为

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

例如，8 位补码 1111 0101 表示 $-128 + 64 + 32 + 16 + 4 + 1 = -11$ 。在 8 位补码中，最大值的编码是 0111 1111，表示 127；最小值的编码是 1000 0000，表示 -128 。一般的 n 位补码取值范围为

$$\boxed{-2^{n-1} \leq x \leq 2^{n-1} - 1}.$$

表 2.5: 同一组 8 位编码的两种整数解释

位串	无符号整数	补码有符号整数
0000 0000	0	0
0000 0001	1	1
0111 1111	127	127
1000 0000	128	-128
1111 0101	245	-11
1111 1111	255	-1

补码的最小值和最大值为什么不对称？因为补码只用一个编码表示零。在 8 位的 256 种组合中，128 种表示负数，127 种表示正数，剩下一一种表示零，所以负数比正数多一个。最小值 -128 可以表示，对应的正数 128 却超出了 8 位有符号补码的范围。读补码时记住最高位的位权为 -128 ，其余各位依次为 64、32、16、8、4、2、1，就能逐位算出结果。

2.5.5 增加位数时的符号扩展

如果把 -11 改用 16 位补码表示，怎样补足新增的 8 位，才能保持数值不变？应当在高位补 1，得到 1111 1111 1111 0101。增加位数时在高位重复原有符号位，称为符

号扩展。如果误补成 0，得到 0000 0000 1111 0101，数值就变成了正数 245。扩展无符号数时，高位补 0；扩展补码有符号数时，高位补原来的符号位。

C 在把较小的有符号整数类型转换为较大的有符号整数类型时，保持原来的数值。下面的赋值把 `signed char` 变量 `small` 转换为 `int`，存入 `wide`：

```
#include <stdio.h>

int main(void) {
    signed char small = -11;
    int wide = small;
    printf("small = %d; wide = %d\n", small, wide);
    return 0;
}
```

程序输出 `small = -11; wide = -11`，说明转换前后数值相同。`small` 的值在传给 `printf` 时会自动转换为 `int`，所以两个参数都用 `%d` 输出。

假设 `signed char` 为 8 位、`int` 为 32 位，两种类型都使用补码，存储位全部用于表示数值和符号。`small` 的编码是 1111 0101；`wide` 的编码在左侧补上 24 个 1，写成十六进制为 `FFFFFFF5`。打印出的十进制值只能验证数值保持不变，位串的变化还需要依据给定的类型位数和补码规则推导。8 位、16 位、32 位的补码长度不同，讨论编码时必须说明位数。

2.6 C 语言中的整数类型

2.6.1 整数类型的长度与符号

表示的数值范围不同，所需的位数也不同。C 提供 `short`、`int`、`long`、`long long`，分别称为短整型、整型、长整型和长长整型。加上 `unsigned`，可以得到相应的无符号整数类型。类型名称有几种等价写法：`short int` 可以简写为 `short`，`long int` 可以简写为 `long`，`unsigned int` 可以简写为 `unsigned`。也可以把长度与符号要求组合起来，写成 `unsigned short` 或 `unsigned long`。

统计一个班的人数，16 位整数已经足够；统计大文件的字节数，就可能需要范围更大的类型。C 允许编译器根据目标机器确定各类型的具体大小，但要求 `short` 占用的空间不大于 `int`，`int` 不大于 `long`，`long` 不大于 `long long`，相邻两种类型可以一样大。常见桌面环境中的 `short` 为 16 位，`int` 为 32 位；`long` 则可能是 32 位，也可能是 64 位。选择类型时，应把待处理数值的范围与类型的实际范围作比较。

表 2.6: 常见位宽下的整数范围（有符号数按补码解释）

有效编码位数	有符号范围	无符号范围
8	-128 ~ 127	0 ~ 255
16	-32768 ~ 32767	0 ~ 65535
32	-2147483648 ~ 2147483647	0 ~ 4294967295

各类型的取值边界可以在 `<limits.h>` 中查到：`INT_MIN` 和 `INT_MAX` 分别给出 `int` 的最小值与最大值，`UINT_MAX` 给出 `unsigned int` 的最大值，`short` 的边界为 `SHRT_MIN`、`SHRT_MAX`，`long` 的边界为 `LONG_MIN`、`LONG_MAX`。C99 还增加了 `long long` 类型，至少能够表示 $-(2^{63} - 1)$ 至 $2^{63} - 1$ 范围内的整数。

2.6.2 常量本身也有类型

变量有类型，直接写在程序中的常量也有类型。21、32、201 采用整数写法，属于整数常量；21.0 带有小数点，3.2E12 采用指数形式，都属于浮点常量。-15 还可以再分成两部分：整数常量 15 和它前面的负号运算符，整个表达式的值为 -15。

对于没有后缀的十进制整数常量，编译器依次检查 `int`、`long int` 和 `long long int`，选择第一个能够表示该数值的类型。没有后缀的八进制和十六进制整数常量依次尝试 `int`、`unsigned int`、`long int`、`unsigned long int`、`long long int`、`unsigned long long int`。例如，0x3 的值为 3，`int` 已经能够表示，所以常量 0x3 的类型就是 `int`。

十进制常量 1000000 的类型也取决于环境。在 `int` 为 16 位的机器上，`int` 的最大值 32767 不足以表示一百万，编译器就要继续检查 `long`；在 `int` 为 32 位的机器上，一百万已经在 `int` 的范围内。常量的写法相同，所在环境不同，最终确定的类型也可能不同。

2.6.3 整数常量的后缀

在整数常量末尾加后缀，可以进一步指定类型的选择范围。`u` 或 `U` 要求使用无符号类型，`l` 或 `L` 要求从长整数类型开始选择，`ll` 或 `LL` 要求从 `long long` 开始选择。符号要求与长度要求可以组合：7LL 的类型为 `long long`，7ULL 的类型为 `unsigned long long`。通常采用大写 L，以免小写 l 与数字 1 混淆。

表 2.7: 整型常量后缀示例

写法	十进制值	常量类型
7u	7	unsigned int
7uL	7	unsigned long int
020L	16	long int
0x10L	16	long int
10lu	10	unsigned long int
123456789L	123456789	long int

常量的后缀与变量定义中的类型名，各自作用于不同的对象。long n = 7; 中的 7 是 int 常量，初始化时转换为 long; int n = 7L; 中的 7L 是 long 常量，初始化时转换为 int。数值 7 在这两种类型中都能准确表示，所以转换不会改变它的值。数值较大时，就要检查目标类型的范围是否足够。

2.6.4 有符号溢出与无符号回绕

如果一个 32 位有符号计数器已经达到 2147483647，再加 1 会怎样？从补码加法器看，只保留低 32 位会得到 80000000，对应 -2147483648。这很容易让人以为，C 程序中的有符号整数到达最大值后也总会这样回到最小值。但 C 将有符号整数溢出规定为未定义行为，这意味着标准不再对程序的行为作出要求，并非只是不确定算式会得到哪个数。编译器还可以在有符号运算不发生溢出的前提下优化代码。因此，计数器接近上限时就应检查，或者事先选用范围足够大的类型。

无符号整数的规则则有明确定义：运算以该类型的最大值加 1 为模。32 位 unsigned int 的最大值是 4294967295，加 1 得到 0，再加 1 得到 1，与前面钟表绕过一整圈的情形相似。下面用 UINT_MAX 取得最大值，观察这一过程：

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    unsigned int j = UINT_MAX;
    printf("%u %u %u\n", j, j + 1u, j + 2u);
    return 0;
}
```

在 32 位 unsigned int 环境中，程序输出 4294967295 0 1。把 -1 转换为 32 位 unsigned int 时，结果也是 UINT_MAX，因为 -1 与 4294967295 对 2^{32} 取模的结果相

同。可见，将变量定义为 `unsigned` 并不能代替输入检查。程序如果只接受非负数，仍应检查使用者是否输入了负号。

2.6.5 整数输出与长度修饰符

输出整数时，格式说明需要与参数类型相匹配。`%d` 表示按十进制输出有符号整数，在 `d` 前加入长度修饰符，就可以指定 `short`、`long` 或 `long long`：

表 2.8: 常用整数输出格式

变量类型	十进制	八进制 / 十六进制（无符号输出）
<code>int</code>	<code>%d</code>	见表后说明
<code>unsigned int</code>	<code>%u</code>	<code>%o</code> / <code>%x</code>
<code>short</code>	<code>%hd</code>	见表后说明
<code>unsigned short</code>	<code>%hu</code>	<code>%ho</code> / <code>%hx</code>
<code>long</code>	<code>%ld</code>	见表后说明
<code>unsigned long</code>	<code>%lu</code>	<code>%lo</code> / <code>%lx</code>
<code>long long</code>	<code>%lld</code>	见表后说明
<code>unsigned long long</code>	<code>%llu</code>	<code>%llo</code> / <code>%llx</code>
<code>size_t</code>	<code>%zu</code>	<code>%zo</code> / <code>%zx</code>

`%o` 和 `%x` 按无符号整数输出，不会输出负号。不带长度修饰符时，两者对应的参数类型都是 `unsigned int`，分别输出八进制和十六进制。C 没有与 `%d` 对应的、直接输出带负号八进制或十六进制整数的格式说明。

如果已有 `int` 变量 `n`，可以用 `(unsigned int)n` 取得一个 `unsigned int` 类型的值，再交给 `%o` 或 `%x`。在表达式前写（类型名），称为**显式类型转换**；转换产生一个指定类型的结果，原变量 `n` 的类型和值保持不变。例如，在函数中写：

```
int n = 26;
unsigned int u = (unsigned int)n;
printf("%d %o %x\n", n, u, u);
```

输出为 `26 32 1a`。这里先进行类型转换：26 由 `int` 转换为 `unsigned int`，数值仍为 26；再进行格式化输出：`%o` 把 26 写成八进制 32，`%x` 把 26 写成十六进制 1a。也可以直接写 `printf("%x", (unsigned int)n);`。格式字符串中的 `%x` 只规定如何输出，类型转换由参数表达式中的 `(unsigned int)` 完成。

负数转换为无符号类型后，数值会改变。例如，在 `unsigned int` 为 32 位的环境中：

```
int n = -1;
unsigned int u = (unsigned int)n;
printf("%d\n", n);
printf("%u\n", u);
printf("%o\n", u);
printf("%x\n", u);
```

四行输出依次为-1、4294967295、3777777777 和 ffffffff。转换按前一小节介绍的取模规则，把 -1 变为 $2^{32} - 1$ ；%o 和 %x 输出的是 4294967295 的八进制和十六进制表示。如果希望把十进制 -26 显示为 -1a，需要由程序单独输出负号，再用 %x 输出大小为 26 的无符号值。

表中 short、long 和 long long 也没有直接输出带负号八进制或十六进制数的格式。需要输出转换后的无符号值时，应分别转换为 unsigned short、unsigned long 和 unsigned long long，再使用表中对应无符号类型的格式。

%hd 由长度修饰符 h 和转换字母 d 组成，合起来才是一项完整的转换说明。单独写 %h，就缺少了转换字母。各字母的大小写也有规定：%ld 中的 l 必须小写；%X 中的大写 X 则表示用大写字母输出十六进制数。

short 作为参数传给 printf 时，会先转换为 int。这种把较小整数类型转换为 int 或 unsigned int 的规则称为**整数提升**；short 的全部取值都能用 int 表示，所以 short 提升为 int。例如，short end = 200；中的 end 既可以用 %hd 输出，也可以在提升后用 %d 输出。long big = 65537L；中的 big 传入后仍是 long，应使用 %ld。如果用 %hd 输出 long 类型的 big，参数类型就不匹配，程序行为未定义；改变输出格式不能代替类型转换。

2.7 浮点数：范围与精度

2.7.1 小数点放在哪里

货币换算得到的 6.22 元可以约定以分为单位，用整数 622 保存。由于单位固定，小数点的位置也就固定了。这种方法适合精确到分的金额计算。物理计算涉及的数值范围往往更广，例如电容可能小到皮法（pF， 10^{-12} 法拉）量级，频率可能高到吉赫（GHz， 10^9 赫兹）量级，固定的小数点位置就不容易兼顾表示范围和精度。

科学计数法提供了另一种思路。例如，70 可以写成

$$70.00 = 0.7 \times 10^2 = 7.0 \times 10^1.$$

把有效数字部分与指数分别保存，就能通过改变指数来调整小数点的位置。浮点数的名

称由此而来。常见计算机采用以 2 为底的表示方法，其基本形式可以写成 $(-1)^s \times m \times 2^e$ ，其中 s 表示符号， m 表示有效数字部分， e 是指数。

浮点数的存储位数有限。1/3 的十进制小数是 $0.333\ldots$ ，只保留前六位就会产生误差。按规定的取舍规则，把一个值改为所用类型能够表示的值，称为舍入。十进制的 0.1 转换成二进制后也是无限循环小数，用有限位数保存时只能取近似值。浮点数能够表示很大范围内的数，却无法表示其中的每一个实数；相邻两个可表示数之间存在间隔。

2.7.2 三种浮点类型

C 提供 `float`（单精度）、`double`（双精度）和 `long double`（长双精度）三种浮点类型，它们能够保存的有效数字位数和表示范围可能不同。IEEE（Institute of Electrical and Electronics Engineers，电气电子工程师学会）制定的 754 标准规定了常用浮点格式和运算规则。在采用 IEEE 754 常见二进制格式的实现中，`float` 占 32 位，`double` 占 64 位。**精度**指能够保留多少位有效数字，**范围**指能表示多大或多小的数。常见情况如下：

表 2.9: 常见二进制浮点类型的精度与范围

类型	常见字节数	十进制有效数字	最大有限值约为
<code>float</code>	4	6 至 7 位	3.4×10^{38}
<code>double</code>	8	15 至 16 位	1.8×10^{308}
<code>long double</code>	依实现而定	不低于 <code>double</code>	不小于 <code>double</code>

`long double` 的具体精度和范围由编译器实现决定。有些实现比 `double` 精度更高，有些则与 `double` 相同。可以通过 `<float.h>` 中的 `FLT_DIG`、`DBL_DIG` 和 `LDBL_DIG` 了解这三种类型的十进制精度，最大有限值分别由 `FLT_MAX`、`DBL_MAX` 和 `LDBL_MAX` 给出。

历史旁注

早期计算机的浮点运算规则各不相同。同一段数值程序换到采用不同浮点格式的机器上，舍入方式和异常情况的处理都可能发生变化。20 世纪 70 年代末，伯克利的卡汉（William Kahan）参与制定统一的浮点算术方案，并参与英特尔 8087 的数据类型与算术设计。8087 是一种协处理器，用来协助主处理器完成浮点计算。1985 年发布的 IEEE 754 标准，对常用浮点格式、舍入方式以及异常值的处理作出了统一规定，使程序员能够更有把握地分析不同机器上的数值结果。卡汉因数值分析方面的贡献获得了 1989 年图灵奖。

有效数字位数与小数位数容易混淆。假设一种表示方法只能保留六位十进制有效数字，0.123456 的六位都在小数点后，123456 的六位则都在小数点前。数值超过六位十进制有效数字能完整记录的整数范围后，固定六位精度就无法逐个表示所有整数，相邻可

表示数的间隔也会随之增大。输出时要求显示十五位小数，只是把已经存下来的近似值打印得更长，并不能增加变量本身的精度。

2.7.3 浮点常量的写法与后缀

浮点常量可以写成小数形式，也可以写成指数形式。19.9、3e5、3.e10 和 .78e5 都是合法写法。指数形式中的 e 或 E 表示乘以 10 的幂，后面必须跟一个十进制整数作为指数，指数前可以带正号或负号。小数点和指数部分至少要有一个；如果写了小数点，它的两侧至少有一侧应当有数字。

按这些规则检查，1.1E 缺少 E 后面的指数，2E3.5 把指数写成了小数，3.5 e5 则用空格隔开了本应连写的常量，三种写法都不合法。3e5 虽然没有小数点，但指数部分完整，因此是合法的浮点常量，数值为 300000。

不带后缀的浮点常量属于 double 类型。后缀 f 或 F 指定 float，后缀 l 或 L 指定 long double：

```
float current = 9.11e9f;
double voltage = 3.3;
long double scale = 4.23e5L;
```

同样的后缀 L，用在 6L 和 6.0L 中，含义并不完全相同。6L 采用整数写法，类型是 long int；6.0L 带有小数点，类型是 long double。判断常量的类型时，需要把主体写法与后缀结合起来看。

2.7.4 类型转换与赋值结果

```
int oranges = 2.99;
float pi = 3.141592653589793;
double ratio = 5 / 2;
double precise_ratio = 5.0 / 2;
```

int oranges = 2.99; 把 oranges 初始化为 2。浮点数转换为整数时，会舍去小数部分，向 0 截断，所以 2.99 变成 2，-2.99 变成 -2；截断后的整数还必须在目标类型的范围内。pi 是 float 变量，右侧却是 double 常量，初始化时需要按 float 的精度舍入。常量写了很多位，并不意味着这些位都能存入 pi。

初始化 ratio 时，先计算 5 / 2。两个操作数都是整数，执行整数除法，商为 2；然后才把这个 2 转换为 double，存入 ratio，结果是 2.0。初始化 precise_ratio 时，5.0 是浮点数，执行的是浮点除法，precise_ratio 因而得到 2.5。表达式先按操作数的类型求值，再把结果转换为目标变量的类型。把左侧变量定义成 double，无法改变右侧已经按整数规则算出的商。

一般的实数计算可以优先使用 `double`。如果需要保存大量数据，再根据存储空间和精度要求考虑 `float`。

2.7.5 加上的 1 到哪里去了

按实数算术， $(2 \times 10^{20} + 1) - 2 \times 10^{20} = 1$ 。先让常量和变量都使用 `double`，观察加法是否保留了 1：

```
#include <stdio.h>

int main(void) {
    double base = 2.0e20;
    double sum = base + 1.0;
    double difference = sum - base;
    printf("%.1f\n", difference);
    return 0;
}
```

在采用 IEEE 754 二进制 64 位 `double`、舍入到最近可表示值的环境中，程序输出 0.0。在 2×10^{20} 附近，相邻两个可表示数的间隔已经大于 1。`base + 1.0` 的精确结果落在两个可表示数之间，舍入后仍等于 `base`，所以 `sum - base` 得到 0。

如果把中间结果存入精度较低的 `float`，赋值还会产生一次舍入。下面的程序仍使用 `double` 常量，但把变量改为 `float`：

```
#include <stdio.h>

int main(void) {
    float a, b;
    b = 2.0e20 + 1.0;
    a = b - 2.0e20;
    printf("%f\n", a);
    return 0;
}
```

实数算术中， $(2 \times 10^{20} + 1) - 2 \times 10^{20} = 1$ 。在采用 32 位 `float`、64 位 `double`，并舍入到最近可表示值的常见二进制实现中，这段程序却输出 4008175468544.000000，约为四万亿。差异来自加法运算和向 `float` 赋值时的两次舍入。

第一步，`2.0e20` 和 `1.0` 都是 `double` 常量。在 2×10^{20} 附近，`double` 能够表示的相邻两个数已经相差不止 1。`2.0e20 + 1.0` 的结果仍被舍入为 `2.0e20`，加上的 1 没有保留下来。

第二步，把加法结果赋给 `float b`，需要再按较低的精度舍入。在 32 位二进制 `float`、舍入到最近可表示值的环境中，`b` 实际保存的值为

200 000 004 008 175 468 544,

比 2×10^{20} 大约 4.008×10^{12} 。

第三步，计算 `b - 2.0e20` 时，`b` 被转换为 `double`。转换保留了 `b` 当前的值，也保留了给 `b` 赋值时产生的误差。`b` 减去 `2.0e20`，得到约 4.008×10^{12} ，正是赋值舍入引入的差值。

如果给混合精度程序中的浮点常量都加上 `f` 后缀，使常量也采用 `float`，在同样的舍入条件下，结果会变成 0，因为相加和相减使用了同一个单精度近似值。全部使用 `double` 的程序已经表明：消除混合精度赋值的误差，也不能保证保留大数加上的 1。分析这类问题时，要逐次检查运算和赋值所用的类型：某一步舍去了精度，后面再改用精度更高的类型，也无法恢复已经丢失的信息。

2.7.6 浮点数输出

输出浮点数时，`%f` 采用小数形式，`%e` 和 `%E` 采用指数形式，`%g` 则根据数值和指定精度选择形式。`float` 传给 `printf` 时会先提升为 `double`，因此 `float` 和 `double` 的值都可以用 `%f` 输出。`long double` 需要使用大写 `L` 作为长度修饰符，例如 `%Lf` 或 `%Le`。

```
float a = 101.56f;
printf("a = %f = %e\n", a, a);
```

在常见的 32 位二进制 `float` 实现中，输出的小数形式为 101.559998，指数形式为 1.015600e+02。`a` 中保存的近似值比 101.56 略小，按小数形式打印时，这一点差异便显现出来；按指数形式打印时，结果又按照默认的小数位数舍入。两次输出取自同一个变量，只是显示方式不同。

2.8 字符、编码与转义序列

2.8.1 用编号表示字符

货币换算程序除了处理数值，还要输出 `RMB` 这样的文字。字母在内存中又怎样表示？早期通信设备就需要解决这个问题：发送端输入字母 `A`，接收端也应当印出 `A`，两端必须对字符的编号作出一致约定。这种编号规则就是**字符编码**。ASCII 使用 0 至 127 这 128 个编号，表示字母、数字、标点以及换行、退格等控制字符。其中 `A` 的编号是 65，写成二进制就是 0100 0001。

表 2.10: 常用 ASCII 编码

字符	十进制	八进制	十六进制
空字符 NUL	0	000	00
空格	32	040	20
0 至 9	48 至 57	060 至 071	30 至 39
A 至 Z	65 至 90	101 至 132	41 至 5A
a 至 z	97 至 122	141 至 172	61 至 7A

ASCII 中的空字符名为 NUL (null character)，编码值为 0，C 用它标记字符串结束。空字符与编码值为 32、显示为一格空白的空格字符不同。`char` 占一个 C 字节，能够存放基本执行字符集中的字符。**执行字符集**是程序运行时使用的字符及其编码约定，基本执行字符集包含 C 要求支持的字母、数字、标点和控制字符。普通 `char` 采用有符号还是无符号的表示方式，由具体实现决定。如果程序明确需要有小整数或无符号的字节值，可以分别使用 `signed char` 和 `unsigned char`。

2.8.2 单引号、双引号与变量名

```
char grade = 'A';
char response;
response = 'B';
```

'A'、A 和"A" 三种写法有不同含义。带单引号的'A' 是字符常量，表示字母 A 的编码值；不带引号的 A 是标识符，可以用作变量名；带双引号的"A" 则是字符串，除了字符 A，还包含末尾的空字符。开头的 `char s[] = "ABCD"`；同样会在四个字母后保存一个空字符，所以数组 `s` 占 5 字节；导言图只画出了四个字母。书写时少了引号或换了一种引号，含义就变了。

`char grade = 'A'`；将字母 A 的编码存入变量 `grade`。C 规定，'A' 这样的普通字符常量属于 `int` 类型，初始化时再转换为 `char`。`long n = 7`；也需要先确定常量类型，再作初始化转换：先确定右侧常量的类型，再按左侧变量的类型进行转换。

在 ASCII 环境中，`char grade = 65`；也能保存字母 A 的编码。不过，直接写'A' 更便于阅读，编译器也会根据执行字符集确定相应的编码值。如果写成 `grade = "A"`；，右侧就成了字符串，无法用这种赋值方式存入单个 `char` 变量。

2.8.3 字符可以参与整数运算

```
#include <stdio.h>

int main(void) {
    char a = 'A';
    char b = a + 25;
    int digit = '8' - '0';
    char encoded = '0' + digit;

    printf("a: %d, %c; b: %d, %c\n", a, a, b, b);
    printf("digit: %d; encoded: %d, %c\n",
           digit, encoded, encoded);

    return 0;
}
```

在 ASCII 环境中运行，得到：

```
a: 65, A; b: 90, Z
digit: 8; encoded: 56, 8
```

a 保存的编码值为 65，用 %d 输出就显示 65，用 %c 输出就显示 A。ASCII 表中，A 到 Z 的编码连续，所以 $b = a + 25$ ；使 b 得到 Z 的编码。 $\text{digit} = '8' - '0'$ 计算 $56 - 48 = 8$ ，把数字字符转换为对应的整数，输出中的 `digit: 8` 直接显示转换结果。 $\text{encoded} = '0' + \text{digit}$ 再计算 $48 + 8 = 56$ ，把整数转换回数字字符的编码；用 %d 打印 encoded 显示 56，用 %c 打印则显示字符 8。C 保证字符 0 至 9 的编码依次连续，因此这种数字字符的减法有语言保证；字母是否连续则取决于字符集，不能在所有编码环境中照用 ASCII 的字母运算。

2.8.4 用转义序列表示控制字符

字母 A 可以直接写在单引号中，换行、制表这样的**控制字符**用于改变输出位置或触发设备动作，不作为普通图形显示。C 采用反斜杠开头的**转义序列**来表示它们，例如用 \n 表示换行，用 \t 表示水平制表。编译器把整个转义序列识别为相应的一个字符。

表 2.11: 本讲使用的转义序列

写法	含义
<code>\n</code>	换行，使后续输出转入下一行
<code>\r</code>	回车，使当前位置回到本行开头
<code>\b</code>	退格，输出位置后退一格
<code>\t</code>	水平制表，移到下一制表位置
<code>\a</code>	警报，是否产生声音由终端决定
<code>\\</code>	一个反斜杠
<code>\'</code>	一个单引号
<code>\"</code>	一个双引号
<code>\0</code>	编码值为 0 的空字符

回车和换行为什么要分成两个字符？早期终端常用电传打字机，计算机的输出直接印在纸上。打完一行之后，需要完成两个动作：让打印位置回到行首，再把纸向前送一行。前一个动作叫 carriage return，后一个动作叫 line feed。后来的屏幕终端沿用了这些控制方式。在 C 中，`\r` 表示回车，使输出位置回到本行开头；`\n` 表示换行，使后续输出转入下一行。文本文件具体采用什么字节序列表示行结束，则由所在系统约定，标准输入输出库会作相应处理。I/O 是 input/output（输入 / 输出）的缩写。

历史旁注

1966 年，兰普森（Butler Lampson）在为 SDS 940 系统编写的讲义中，专门说明了电传打字机的输入、输出和回显过程。用户按下一个字母键，系统接收这个字符后，再将它发回终端，纸上才印出相应的字母。系统还可以发送控制字符，让终端响铃、回车或送纸换行。今天在终端中输入命令时，屏幕会显示刚刚键入的文字，这种回显方式延续了早期终端的交互习惯。回车、换行等名称，也保留了当年打印机构的工作特点。

退格 `\b` 使输出位置后退一格，通常不会直接擦去原有字符，后续输出才可能覆盖退格位置原有的字符。水平制表 `\t` 使输出位置移到下一个制表位，即终端预先设定的对齐列，移动的距离取决于当前所在的列。双引号和反斜杠在 C 中另有语法用途，要在字符串中表示它们本身，也需要转义：

```
printf("if \"you\" aren't right\\n");
```

这条语句输出 `if "you" aren't right\`，随后换行。其中，`\` 表示双引号，`\\` 表示反斜杠，末尾的 `\n` 表示换行。字符串中的单引号可以直接书写，所以 `aren't` 里的单引号不必转义。

2.8.5 八进制与十六进制转义

知道字符的编码值后，还可以用八进制或十六进制转义序列表示它。八进制写法是在反斜杠后接 1 至 3 个八进制数字，例如 `'\040'`；十六进制写法以 `\x` 开头，例如 `'\x20'`。在 ASCII 环境中，这两个字符常量的值都是十进制 32，都表示空格：

表 2.12: ASCII 环境中的等价字符表示

字符	通常写法	八进制转义	十六进制转义
空格	<code>' '</code>	<code>'\040'</code>	<code>'\x20'</code>
水平制表	<code>'\t'</code>	<code>'\011'</code>	<code>'\x09'</code>
小写 a	<code>'a'</code>	<code>'\141'</code>	<code>'\x61'</code>
逗号	<code>','</code>	<code>'\054'</code>	<code>'\x2C'</code>

`'\40'` 和 `'\040'` 分别用了两位和三位八进制数字，表示的值相同，都是十进制 32。十六进制转义必须用小写 `x` 开头，后面的十六进制字母则可以大写或小写。与八进制转义最多读取三位数字不同，十六进制转义会一直读取后续的十六进制数码，直到遇见不属于数码的字符。因此，在字符串中书写时，要防止后面的 A 至 F 也被并入转义序列。

这类数值转义的值必须在 `unsigned char` 的表示范围内，表中的例子都选自 ASCII 的 0 至 127。对于换行、制表等常用控制字符，直接写 `'\n'`、`'\t'` 通常更清楚，也不必记忆编码值；换用其他执行字符集时，编译器会确定相应的编码。

2.8.6 一个字符会占几个字节

ASCII 的 128 个编号容纳不了中文、希腊文和阿拉伯文所需的大量字符。Unicode（统一码）为字符分配称为码点的编号，例如字母 A 的码点为 U+0041，“中”的码点为 U+4E2D；U+ 后面的数字按十六进制书写。

UTF 是 Unicode Transformation Format（Unicode 转换格式）的缩写，规定如何把 Unicode 码点存成二进制。UTF-8 以 8 位为一个编码单位，可以用不同数量的字节保存不同的字符。

在字节为 8 位、文本采用 UTF-8 编码的环境中，字母 A 编码为 41，占 1 字节；“中”编码为 E4 B8 AD，占 3 字节。一个 `char` 只能保存其中一个字节，保存“中”需要连续的三个字节。因此，文本占用的字节数未必等于字符数。只用基本英文字符测试程序时，字节数与字符数恰好相同，便不容易发现差别。

其他 Unicode 编码形式（选读）

编码单元是组织编码的基本单位。UTF-8、UTF-16、UTF-32 的编码单元分别为 8 位、16 位和 32 位。UTF-8 用 1 至 4 个编码单元，UTF-16 用 1 或 2 个编码单元，

UTF-32 用 1 个编码单元表示一个 Unicode 标量值。**标量值**指除去代理码点的码点；**代理码点**是 Unicode 为 UTF-16 成对编码保留的一段编号，不单独表示字符。码点是字符的编号，编码单元是存储编号所用的单位，不能据编码单元的名称推断一个字符只占一个字节。

2.9 格式化输出：把值变成文本

2.9.1 格式字符串与参数的对应关系

程序算出了结果，还需要把结果与说明文字一起输出。`printf` 的第一个参数是**格式字符串**，其中既可以写普通文字，也可以写转换说明。普通文字按原样输出，转换说明则与后面的数据参数依次对应。例如，输出一次加法的算式：

```
int a = 1, b = 2;
printf("%d + %d = %d\n", a, b, a + b);
```

第一个`%d` 对应 `a`，第二个对应 `b`，第三个对应表达式 `a + b` 的值，最终输出 `1 + 2 = 3`。其中的空格、加号和等号，都是格式字符串中直接写出的字符。

`printf` 允许传入数量不固定的参数，需要根据格式字符串确定怎样使用这些参数。遇到`%d`，就按 `int` 类型处理对应参数；遇到`%f`，就按 `double` 类型处理。格式说明与实际参数的类型必须相符。若传入一个 `double`，却用`%d` 要求按整数输出，就违反了这一规则。对于直接写在代码中的格式字符串，编译器通常能够检查出此类问题并给出警告，应当逐项修正。

2.9.2 字段宽度、精度与百分号

输出一列金额时，除了保留几位小数，还要考虑对齐。`%8.2f` 表示采用小数形式，小数点后保留两位，整个输出字段至少占 8 个字符；加上减号，写成`%-8.2f`，则改为左对齐。转换说明的各部分按以下顺序书写，方括号内的部分可以省略：

`%[标志][宽度][. 精度][长度修饰符] 转换字母.`

```
double value = 12.5;
printf("|%8.2f|\n", value);
printf("|%-8.2f|\n", value);
printf("progress: %d%%\n", 75);
```

输出为：

```
| 12.50|  
|12.50 |  
progress: 75%
```

竖线标出了字段的边界。12.50 占五个字符，距离规定的宽度 8 还差三个字符，所以用三个空格补齐。宽度规定的是最小值，如果输出内容超过八个字符，字段会自动加宽，数值不会被截断。%.2f 只对显示结果进行舍入，不会改变变量中保存的值。%g 的精度含义有所不同，指定的是有效数字位数。要输出百分号本身，需在格式字符串中写%%，因此最后一行会显示 75%。

2.9.3 三类参数不匹配

这三个 printf 调用依次存在多传参数、少传参数、类型与顺序不符的问题：

```
int f = 4;  
int g = 5;  
float h = 5.0f;  
  
printf("%d\n", f, g);  
printf("%d %d\n", f);  
printf("%d %f\n", h, g);
```

第一个 printf 调用只有一个%d，使用参数 f，所以输出 4。多余的参数 g 仍会被求值，但没有对应的转换说明，printf 不会输出它。删除第一个调用中多余的 g，就能使参数与转换说明一一对应。

第二个 printf 调用有两个%d，却只提供了一个数据参数，缺少第二个参数会导致未定义行为。第三个 printf 调用的参数数量虽然足够，类型却与位置不符：h 提升为 double 后对应了%d，int 类型的 g 又对应了%f，同样会导致未定义行为。程序可能打印出无意义的数，也可能发生其他错误。检查时，可以从左到右将每项转换说明与对应参数逐一核对。

三个 printf 调用分别改为：

```
printf("%d\n", f);  
printf("%d %d\n", f, g);  
printf("%f %d\n", h, g);
```

如果需要按整数形式输出 h，可以先进行类型转换。例如，printf("%d", (int)h); 中的 (int) 执行显式类型转换，转换后的结果再由%d 输出。转换所得的整数必须在 int 的表示范围内。

2.10 格式化输入与交互过程

2.10.1 将输入文本转换为数值

换算程序中只输入了一个整数。一次调用 `scanf` 也可以输入多个数据，例如同时读入一个计数值和一个电压值：

```
int count;
double voltage;
scanf("%d %lf", &count, &voltage);
```

输入 12 3.3 时，`scanf` 先按 `%d` 将 12 转换为 `int`，存入 `count`，再按 `%lf` 将 3.3 转换为 `double`，存入 `voltage`。两个地址 `&count` 和 `&voltage` 分别指定数据的存放位置。**标准输入**是程序默认的输入来源，用 `stdin` 表示，交互运行时通常连接键盘。将输入来源从键盘改成文件，或将输出目标从屏幕改成文件，称为**重定向**。

表 2.13: 常用类型的输入与输出对照

变量类型	<code>printf</code> 的格式与参数	<code>scanf</code> 的格式与参数
<code>int n</code>	<code>%d, n</code>	<code>%d, &n</code>
<code>unsigned int n</code>	<code>%u, n</code>	<code>%u, &n</code>
<code>short n</code>	<code>%hd, n</code>	<code>%hd, &n</code>
<code>long n</code>	<code>%ld, n</code>	<code>%ld, &n</code>
<code>float x</code>	<code>%f, x</code>	<code>%f, &x</code>
<code>double x</code>	<code>%f, x</code>	<code>%lf, &x</code>
<code>long double x</code>	<code>%Lf, x</code>	<code>%Lf, &x</code>
<code>char ch</code>	<code>%c, ch</code>	<code>%c, &ch</code>

输入浮点数时，`float` 需要用 `%f`，`double` 需要用 `%lf`，与输出时的规则不同。原因在于，输出传递的是变量的值，`float` 可以在调用函数前提升为 `double`；输入传递的是变量的地址，变量本身的类型和存储空间已经确定，函数必须按照这个类型写入数据。`float` 变量的地址不会因为传给了函数就变成 `double` 变量的地址。同样，向 `short` 变量输入整数时，应当使用 `%hd`。

2.10.2 检查输入是否成功

`scanf` 的返回值说明成功转换并赋值了多少个数据。读入计数值 `count` 和电压值 `voltage` 需要完成两次转换，因此应检查返回值是否等于 2：

```
if (scanf("%d %lf", &count, &voltage) != 2) {  
    printf("Invalid input.\n");  
    return 1;  
}
```

如果输入 12 abc，第一个数 12 已经存入 count，第二次转换却在字母 a 处失败，voltage 没有得到新值，函数返回 1。如果在第一次转换之前就遇到输入结束或输入故障，函数会返回 EOF。EOF 是 end of file（文件结束）的缩写，是 <stdio.h> 定义的负整数宏；scanf 也用 EOF 报告首次转换前的输入故障。检查返回值是否等于 2，可以保证 count 和 voltage 都获得新值后，程序才继续计算。

如果输入 12 3.3abc，两个数值转换都能完成，返回值仍为 2；后面的 abc 尚未读取。返回值只统计成功赋值的数量，不能证明整行输入没有多余字符。是否接受后续字符，要由程序的输入规则另行判断。

输入值还必须在目标类型的表示范围内。例如，向 32 位 int 输入一个远大于其最大值的整数，不能只靠检查返回值来处理。本讲示例假定输入的数值均在相应类型的范围内。需要处理任意用户输入的程序，通常应先读取整行文本，再使用能够报告转换错误和范围错误的函数进行解析。

2.10.3 空白、普通字符与字符输入

输入两个数时，可以用空格隔开，也可以按 Enter 换行。%d、%f 等数值转换会跳过数据前面的空白字符。如果格式字符串中写了普通字符，输入中就必须相应位置出现同样的字符。例如，scanf("%d,%d", &a, &b)；要求两个数之间有逗号：输入 1,2 能够成功读入，输入 1 2 则会在匹配逗号时失败，无法继续读入第二个数。

字符输入容易遇到另一种情况：程序先要求输入数字，再要求输入字母，使用者刚输入完数字并按下 Enter，程序就跳过了等待字母的步骤。这通常是因为%c 读入了前一次输入留下的换行符。%c 不会跳过空白，空格和换行也会作为普通字符读入。如果要读取下一个非空白字符，可以在%c 前加一个空格：

```
scanf(" %c", &ch);
```

格式" %c" 中，%c 前面的空格会匹配并跳过连续的空白字符，然后才由%c 读入一个字符。如果要读的本来就是空格或换行，则应使用不带前导空格的%c。

还有一种容易误写的形式是 scanf("%d\n", &n)。使用者输入整数并按下 Enter 后，程序可能仍在等待输入，因为格式字符串末尾的\n 要求继续跳过空白，直到遇到非空白字符或输入结束。格式字符串末尾的\n 并不表示“按一次 Enter 就结束”。只想读入一个普通整数时，格式字符串写%d 即可。

2.10.4 月薪计算：观察控制字符的作用

月薪计算程序用退格、制表和回车控制文字的位置。程序先显示七条下划线，标出输入金额的位置；读入期望月薪后，按 12 个月计算年薪，并在结果所在行的开头加上 Gee!：

```
#include <stdio.h>

int main(void) {
    float salary;

    printf("\aEnter your desired monthly salary: ");
    printf("$_____\b\b\b\b\b\b\b");
    fflush(stdout);
    if (scanf("%f", &salary) != 1) {
        return 1;
    }
    printf("\n\t$%.2f a month is $%.2f a year.",
           salary, salary * 12);
    printf("\rGee!\n");
    return 0;
}
```

第一条提示语中的 \a 用于请求警报，终端可能响铃，也可能采用其他提示方式。随后，程序输出美元符号和七条下划线，再用七个 \b 把输出位置退回第一条下划线处。在通常的终端回显方式下，输入 345 会覆盖前三条下划线，同时 salary 读入数值 345。乘以 12 后得到 4140，两个 %.2f 分别把月薪和年薪显示为 345.00 与 4140.00。

输出结果时，\n\t 先换行，再移到下一个制表位，从制表位开始打印金额说明。后面的 \r 又把输出位置移回本行开头，使 Gee! 出现在行首，最后再换行。如果把程序输出重定向到文件，文件中会保存这些控制字符；打开文件时，它们产生怎样的显示效果，取决于软件的解释方式。

2.10.5 输出何时真正出现

有时，程序已经执行了 printf，屏幕上却暂时没有文字，等到输入或程序退出时才显示出来。这与标准 I/O 库的缓冲区有关。输出内容可以先暂存在缓冲区中，积累到一定程度后再一起送往外部设备。这样做是因为外部写入有开销，逐字符写入的效率较低，集中写入能够减少调用次数。

标准输出连接交互终端时，通常采用行缓冲，遇到换行就将这一行送出；缓冲区写

满时也会送出，相关的输入操作还可能触发刷新。标准输出重定向到文件后，通常改用全缓冲，积累更多内容后才写入文件。缓冲方式随输出目标而变化，因此同一个程序在终端显示和文件输出时，可能表现出不同的刷新时机。

月薪程序调用 `fflush(stdout)` 刷新标准输出，使提示语在等待输入前显示。程序从 `main` 正常返回时，标准库也会处理尚未刷新的输出。`fflush` 用于刷新输出流；标准 C 并未规定可以用 `fflush(stdin)` 清空键盘输入。

2.11 习题与解析

九组练习分别检验常量与格式的识别、程序改错、补码解释、浮点计算和输入过程。做改错题时，可以先找语法错误，再检查程序的计算过程：变量使用前是否已经赋值，类型转换是否丢掉了所需的小数，格式说明是否与参数相符。

2.11.1 习题一：判断常量写法

判断 `E-4`、`A423`、`0xABCL`、`1.1E`、`003` 与 `-2E3.5` 各属于哪种写法，并指出其中不合法的形式。

表 2.14: 常量写法判断

写法	判断与理由
<code>E-4</code>	不是一个数值常量；若 <code>E</code> 已定义，可以构成 <code>E</code> 减 4 的表达式。
<code>A423</code>	是合法的标识符写法，不是数值常量。
<code>0xABCL</code>	合法的十六进制长整数常量， <code>L</code> 是后缀；值为 2748。
<code>1.1E</code>	不合法的浮点常量写法， <code>E</code> 后缺少指数数字。
<code>003</code>	合法的八进制整数常量，值为 3，类型为 <code>int</code> 。
<code>-2E3.5</code>	写法不合法，浮点常量的指数不能含小数点。

以 `0xABCL` 为例，`0x` 是十六进制前缀，`A`、`B`、`C` 依次表示 10、11、12，最后的 `L` 是长整数后缀。所以 `0xABCL` 的值为 $10 \times 16^2 + 11 \times 16 + 12 = 2748$ 。`003` 则由开头的 0 确定为八进制，后面的数码均合法，数值为 3。

2.11.2 习题二：常量的类型、意义与转义

表 2.15: 常量含义

常量	C 中的类型	含义
'\b'	int	普通字符常量，表示退格
1066	int	十进制整数 1066
99.44	double	数值 99.44 的浮点表示
0XAA	int	十六进制 AA，即十进制 170
2.0e30	double	2.0×10^{30} 的浮点表示

常用转义序列 `\n`、`\\`、`\"`、`\t` 依次表示换行、反斜杠、双引号和水平制表。将转义序列放在单引号中，就构成一个字符常量；在字符串中使用时，则直接写在双引号包围的内容中。

2.11.3 习题三：修正税额计算程序

某程序要按给定税率计算税额，却写成了下面的样子。先找出妨碍编译的语法错误，再检查计算税额所需的数据是否已经赋值：

```
include <stdio.h>
main
(
    float g; h;
    float tax, rate;

    g = e21;
    tax = rate * g;
)
```

第一行的 `include` 前缺少 `#`。主函数应写成 `int main(void)`，函数体应当用花括号包围。`float g; h;` 中的分号结束了 `g` 的声明，后面的 `h` 并没有被定义；如果需要两个浮点变量，应写成 `float g, h;`。`e21` 会被识别为标识符，要表示 10^{21} ，必须在 `e` 前写出有效数字部分，改成 `1e21`。

修正语法后，还要检查 `tax = rate * g;` 中的 `rate`。程序虽然定义了这个变量，却没有给出税率，乘法便使用了尚未赋值的数据。若设 `g = 1e21f`、`rate = 0.17f`，并删除未使用的 `h`，可以得到下面的程序：

```
#include <stdio.h>

int main(void) {
    float g;
    float tax, rate;

    g = 1e21f;
    rate = 0.17f;
    tax = rate * g;
    printf("tax = %.3e\n", tax);
    return 0;
}
```

程序输出约为 `tax = 1.700e+20`。括号、分号等语法错误通常会在编译时被指出；变量是否在使用前获得了正确的值，还需要结合程序的执行过程检查。乘法 `rate * g` 要求税率 `rate` 和计税金额 `g` 都已经获得数值，定义变量本身没有给出税率和金额。

2.11.4 习题四：由牛腿数计算牛的数量

假设每头牛都有四条腿，知道腿的总数，除以 4 就能算出牛的数量。下面的程序用于完成这个计算，但在注释、变量定义和输入输出语句中都有错误：

```
void main(int) { / This program is perfect /
    cows, legs integer;

    printf("How many cow legs did you count?\n");
    scanf("%c", legs);
    cows = legs / 4;
    printf("That implies there are %f cows.\n", cows);
}
```

主函数应写为 `int main(void)`，注释应放在 `/* ... */` 之间。牛的数量与腿的数量都用整数表示，因此定义为 `int cows, legs`；。输入 `legs` 时，使用 `%d`，并传入地址 `&legs`；输出 `cows` 时，同样使用 `%d`。再补上头文件与返回语句，程序如下：

```
#include <stdio.h>

int main(void) {
    int cows, legs;

    printf("How many cow legs did you count?\n");
    if (scanf("%d", &legs) != 1) {
        return 1;
    }
    cows = legs / 4;
    printf("That implies there are %d cows.\n", cows);
    return 0;
}
```

输入 20，程序算出 5 头牛；输入 22，结果也为 5，因为整数除法会舍去余数。后一个结果说明还需要检查输入是否符合题意。按每头四条腿计算，完整牛群的腿数应当非负，而且能被 4 整除。如果程序要核查牛群记录，就应在计算前检查腿数非负且能被 4 整除。语法与格式正确，只能保证程序按所写的规则执行，输入是否合理仍需另作判断。

2.11.5 习题五：变量类型与输出格式

为下表各常量选择合适的变量类型，并写出该变量的输出格式。表中同时列出常量自身的类型，以便区分常量类型、初始化时的转换和变量的输出方式。

表 2.16: 习题五：常量类型与使用方式

常量	常量的类型	可选变量类型	输出格式
12	int	int	%d
0x3	int	unsigned int	%#x
'C'	int	char	%c
2.34E07	double	double	%E
'\040'	int	char	%c
7.0	double	double	%f
6L	long int	long	%ld
6.0f	float	float	%f

例如，`unsigned int n = 0x3`；先把值为 3 的 `int` 常量转换为 `unsigned int`，再存入 `n`。用 `%#x` 输出这个变量，得到 `0x3`。如果改用 `int n = 0x3`；再以 `%d` 输出，得

到的就是 3。'C' 和 '\040' 都是 int 类型的字符常量，存入 char 变量时发生类型转换；在 ASCII 环境中，'\040' 的编码对应空格。

2.11.6 习题六：前缀、后缀与类型选择

表 2.17: 习题六：常量类型与使用方式

常量	常量的类型	可选变量类型	输出格式
012	int	unsigned int	%#o
2.9e05L	long double	long double	%Le
's'	int	char	%c
100000	见下文	long	%ld
'n'	int	char	%c
20.0f	float	float	%f
0x44	int	unsigned int	%#x

012 表示十进制 10，0x44 表示十进制 68。字符常量 'n' 表示字母 n，'\n' 表示换行，输出时两者的效果不同。2.9e05L 采用浮点写法并带有 L 后缀，类型为 long double，可用 %Le 输出。

在 32 位 int 环境中，常量 100000 的类型为 int；如果 int 的范围不足以表示它，编译器会继续尝试 long。定义 long n = 100000；可以在这两种情况下保存该数值，并用 %ld 输出。若直接写成 100000L，后缀就指定了从长整数类型开始选择。

2.11.7 习题七：同一位串的解释与扩展

给定 8 位位串 1111 0110，完成下列问题：

1. 分别按无符号整数和补码有符号整数解释，求出数值。
2. 将补码扩展为 16 位并保持数值不变，写出位串。
3. 如果误在高位补 0，16 位补码表示的数值是多少？

解析。按无符号数解释，数值为 $128 + 64 + 32 + 16 + 4 + 2 = 246$ 。按 8 位补码解释，数值为 $246 - 256 = -10$ 。扩展为 16 位时，高位应补 1，得到 1111 1111 1111 0110；按负位权计算仍为 -10 。若高位补 0，得到 0000 0000 1111 0110，16 位补码的符号位为 0，数值变成正数 246。高位补什么，取决于位串采用的解释规则。

2.11.8 习题八：计算精度与输出精度

预测下面两行输出，并说明把%.6f 改为%.12f 能否让 x 显示成 2.5:

```
double x = 5 / 2;
double y = 5.0 / 2;
printf("x = %.6f\n", x);
printf("y = %.6f\n", y);
```

解析。输出依次为 $x = 2.000000$ 和 $y = 2.500000$ 。 $5 / 2$ 按整数除法得到 2，再转换为 double； $5.0 / 2$ 按浮点除法得到 2.5。增加输出小数位数，只会让 x 显示为 2.000000000000，无法改变已经计算出的值。

再考虑第2.7节的大数实验：在 64 位二进制 double、舍入到最近可表示值的环境中，base 为 $2.0e20$ ，为什么 $sum = base + 1.0$ 之后的 $sum - base$ 为 0？因为加 1 后的精确值无法用当前精度表示，舍入结果仍为 base。整数除法舍去的余数与浮点加法舍入丢失的增量都发生在计算时，增加显示位数不能恢复。

2.11.9 习题九：连续输入与剩余字符

运行下面的程序，输入 12 A 后按 Enter。写出 first、second、n 和 ch 的值，并说明还有哪些字符没有被读取。再把第二次调用中的"%c" 改为" %c"，重新运行并输入相同内容，比较结果。

```
#include <stdio.h>

int main(void) {
    int n = 0;
    char ch = '?';
    int first = scanf("%d", &n);
    int second = scanf("%c", &ch);
    printf("first=%d, second=%d, n=%d, ch=[%c]\n",
           first, second, n, ch);
    return 0;
}
```

解析。第一次调用读取 12 后停止，空格、A 和换行仍留在输入中。%c 不跳过空白，所以第二次调用读入空格。两个返回值都是 1，n 为 12，ch 为空格，程序输出：

```
first=1, second=1, n=12, ch=[ ]
```

字母 A 和换行还未被读取。改用" %c" 后，格式中的前导空格跳过输入中的空格，%c 随后读取 A；输出末尾变为 $ch=[A]$ ，换行还未被读取。两次调用都返回 1，并不表示整行输入已经全部读取。

2.12 本讲总结

开头的货币换算程序中，使用者输入 100，程序算出 622.00，其中经历了字符与数值的转换、浮点乘法和输出时的小数位数选择。`scanf` 根据格式说明转换输入字符，把数值存入指定地址的变量；`printf` 根据格式说明，将变量的值转换为可以阅读的文字。

整数能够表示多大的数，取决于位数和编码方式。理解了位权，就能进行进制转换；利用三位二进制对应一位八进制、四位二进制对应一位十六进制的关系，还可以方便地查看编码。补码使普通二进制加法器能够完成有符号数的加减。写 C 程序时，还应区分语言对两类整数的规定：有符号整数溢出会导致未定义行为，无符号整数则按模运算规则回绕。

浮点数通过有效数字部分和指数表示数值，在有限位数下兼顾范围与精度，因此存储和运算中都可能发生舍入。大数加上一个很小的数，结果可能仍然等于原来的近似值；赋给精度较低的变量，还可能产生新的误差。遇到计算结果与手算不符时，应按执行顺序检查各步运算的类型，以及赋值过程中发生的转换。

字符在内存中保存为编码值。在 ASCII 环境中，'A' 对应 65，既可以用 `%c` 输出字母，也可以用 `%d` 输出编码。换行、回车、退格等控制字符用转义序列书写，它们的用途可以追溯到早期的打印终端。处理中文等文本时，还要考虑一个字符可能占用多个字节，不能把字节数直接当作字符数。

编写输入输出语句时，可以逐项核对：变量的类型与格式说明是否相符，输入函数是否成功读到了所需的数据，结果的小数位数和排列方式是否合适。